

Prueba Técnica — Java Junior

API REST de gestión de pagos

1. Contexto

Una plataforma de pagos necesita un servicio que permita a los comercios crear y consultar órdenes de pago. El objetivo es desarrollar una API REST sencilla que permita gestionar el ciclo básico de una transacción.

2. Tecnologías

- Java 17 o superior
- Spring Boot
- Maven o Gradle
- Spring Data JPA
- H2 o PostgreSQL
- JUnit 5
- Git

3. Requerimientos funcionales

3.1 Crear una orden de pago

POST /api/payments

Request:

```
{ "merchantId": "MERCHANT-001", "amount": 150.50, "currency": "PAB", "description":  
"Compra #12345", "customerEmail": "customer@example.com" }
```

El sistema debe generar un ID único para el pago, guardar la fecha de creación y establecer inicialmente el estado **PENDING**. Los datos recibidos deben ser validados.

Estados posibles: PENDING, APPROVED, DECLINED, CANCELLED

3.2 Consultar un pago

GET /api/payments/{id}

Response de ejemplo:

```
{ "id": "PAY-123456", "merchantId": "MERCHANT-001", "amount": 150.50, "currency":  
"PAB", "description": "Compra #12345", "customerEmail": "customer@example.com",  
"status": "PENDING", "createdAt": "2026-09-14T15:30:00" }
```

Si el pago no existe, la API debe responder con **404 Not Found**.

3.3 Actualizar el estado de un pago

PATCH /api/payments/{id}/status

Request de ejemplo:

```
{ "status": "APPROVED" }
```

El sistema debe validar que la transición de estado sea válida. Se permiten, como mínimo, las siguientes transiciones:

Estado actual	Estados permitidos
PENDING	APPROVED, DECLINED, CANCELLED
APPROVED	Ninguno
DECLINED	Ninguno
CANCELLED	Ninguno

Por ejemplo, no debería permitirse cambiar un pago **APPROVED** a **PENDING** o **DECLINED**, ni cambiar un pago **CANCELLED** a **APPROVED**.

3.4 Consultar pagos de un comercio

GET /api/merchants/{merchantId}/payments

Debe permitir obtener los pagos asociados a un comercio y, como mínimo, filtrar por estado.

```
GET /api/merchants/MERCHANT-001/payments?status=APPROVED
```

3.5 Resumen del comercio

GET /api/merchants/{merchantId}/summary

Response de ejemplo:

```
{ "merchantId": "MERCHANT-001", "totalPayments": 25, "approvedPayments": 20,
  "declinedPayments": 3, "cancelledPayments": 2, "totalApprovedAmount": 1250.75 }
```

4. Requisitos técnicos

Arquitectura

Se espera una separación razonable de responsabilidades. Como referencia, pueden utilizarse capas como **controller, service, repository, entity, dto y exception**. No es necesario implementar una arquitectura compleja.

Manejo de dinero

Los montos deben manejarse correctamente. **No utilizar tipos de punto flotante para representar dinero.**

Validaciones

- merchantId obligatorio.
- amount mayor a 0.
- currency obligatoria.
- description obligatoria.
- email con formato válido.
- status válido.

Manejo de errores

Código	Situación
400 Bad Request	Datos inválidos
404 Not Found	Pago inexistente
409 Conflict	Operación no permitida
500 Internal Server Error	Error inesperado

Seguridad

No es necesario manejar números de tarjeta, CVV, PAN ni otros datos sensibles de pago. La prueba debe trabajar únicamente con los datos definidos en este documento.

5. Tests

Implementar como mínimo **3 tests**, incluyendo al menos 2 tests unitarios y 1 test de integración de un endpoint.

- Algunos escenarios que pueden considerarse: Transición PENDING → APPROVED válida.
- Algunos escenarios que pueden considerarse: Transición PENDING → DECLINED válida.
- Algunos escenarios que pueden considerarse: Transición APPROVED → DECLINED inválida.
- Algunos escenarios que pueden considerarse: Validación de amount ≤ 0 .
- Algunos escenarios que pueden considerarse: Consulta de un pago inexistente.

6. Entrega

La solución debe entregarse en un repositorio Git. El README.md debe incluir:

- Instrucciones para ejecutar el proyecto.
- Instrucciones para ejecutar los tests.

- Endpoints disponibles.
- Ejemplos de requests.
- Decisiones técnicas relevantes.
- Cualquier supuesto realizado durante la implementación.

7. Tiempo

La prueba tiene un plazo de **2 a 3 días** para su entrega y está pensada para aproximadamente **4 a 6 horas de trabajo efectivo**. No es necesario dedicar tiempo adicional a funcionalidades que no estén solicitadas.

8. Consideraciones

Podés utilizar documentación oficial, buscadores y herramientas de asistencia al desarrollo durante la prueba. Lo importante es que puedas comprender y explicar las decisiones tomadas y el código entregado.